Projet Cloud Healthcare Unit : présentation, fonctionnement et stack technique

1. Contexte et objectifs

Le groupe hospitalier CHU nous confie la conception et la réalisation d'un entrepôt de données décisionnel capable de répondre aux questions de ses praticiens et de sa direction. Le projet est réalisé en groupe de trois sur les trois semaines du bloc « Stockage et Big Data », et chaque boucle PBL fait avancer une brique de la solution.

Aujourd'hui, les données du groupe sont dispersées : une base opérationnelle PostgreSQL pour les actes médicaux, des fichiers CSV pour le référentiel des établissements, des fichiers déposés sur un serveur FTP pour la satisfaction des patients et les décès. Aucune de ces sources ne permet, seule, de calculer un taux d'hospitalisation par diagnostic ou une satisfaction moyenne par région.

Notre objectif est de construire une plateforme qui :

- collecte automatiquement ces quatre sources et les conserve dans un stockage unique, historisé et gouverné ;
- nettoie, harmonise et pseudonymise les données pour qu'elles soient fiables et conformes au RGPD ;
- les organise dans un modèle décisionnel en étoile optimisé pour l'analyse ;
- les restitue dans une application de visualisation que nous développons nous mêmes, adaptée aux deux profils d'utilisateurs.

Nous avons fait le choix d'une architecture réellement Big Data, fondée sur Apache Spark et un Lakehouse Delta Lake, plutôt que la stack Cloudera et Talend proposée dans le manuel d'installation. Ce choix est justifié dans la section 5.

2. Ce qui est demandé

Le sujet impose huit indicateurs métier, trois livrables et des exigences non fonctionnelles strictes. Tout ce que nous construisons doit servir l'un de ces trois points.

Indicateurs attendus

Le mot « taux » n'est pas défini dans le sujet. Nous fixons pour chaque indicateur un numérateur et un dénominateur, documentés dans le dictionnaire de données.

N°	Indicateur	Définition retenue	Statut
1	Taux de consultation par établissement sur une période	Consultations de l'établissement / consultations totales de la période	Obligatoire
2	Taux de consultation par diagnostic sur une période	Consultations portant le diagnostic / consultations totales de la période	Obligatoire
3	Taux global d'hospitalisation sur une période	Patients hospitalisés / patients suivis sur la période	Obligatoire
4	Taux d'hospitalisation par diagnostic	Hospitalisations portant le diagnostic / hospitalisations totales	Obligatoire
5	Taux d'hospitalisation par sexe et par âge	Répartition des hospitalisations par sexe et tranche d'âge à l'admission	Obligatoire
6	Taux de consultation par professionnel	Consultations du professionnel / consultations totales	Obligatoire
7	Nombre de décès par région en 2019	Comptage des décès par région de décès	Optionnel, réalisé
8	Taux global de satisfaction par région en 2020	Score moyen de satisfaction des établissements de la région	Optionnel, réalisé

Nous traitons aussi les deux indicateurs optionnels : ils apportent la dimension géographique et les cartes qui rendront la soutenance plus parlante.

Livrables

Livrable	Format	Contenu principal
L1 Référentiel de données et architecture	Rapport	Modèles conceptuel et logique, dictionnaire de données, architecture Bronze Silver Gold, spécifications ETL, plan de gouvernance

Livrable	Format	Contenu principal
L2 Implémentation physique et performance	Rapport + archive ZIP	Scripts de création et de chargement, partitionnement, rapport qualité, au moins 5 requêtes de benchmark, graphiques avant et après optimisation
L3 Restitution et storytelling	Soutenance 20 min + 10 min de questions	Tableau de bord interactif, récit pour praticiens et direction, démonstration live, recommandations

Exigences non fonctionnelles

- Ratio coût efficacité : uniquement des briques open source, déployables sans licence.
- Sécurité des données de santé : conformité RGPD et trajectoire vers un hébergement certifié HDS.
- Élasticité et scalabilité : une architecture qui passe d'un poste local à un cluster sans réécriture.

3. Les sources de données

Les quatre sources sont imposées et fournies dans le pack DATA 2024. Notre travail consiste à les intégrer telles quelles, sans les modifier à la source.

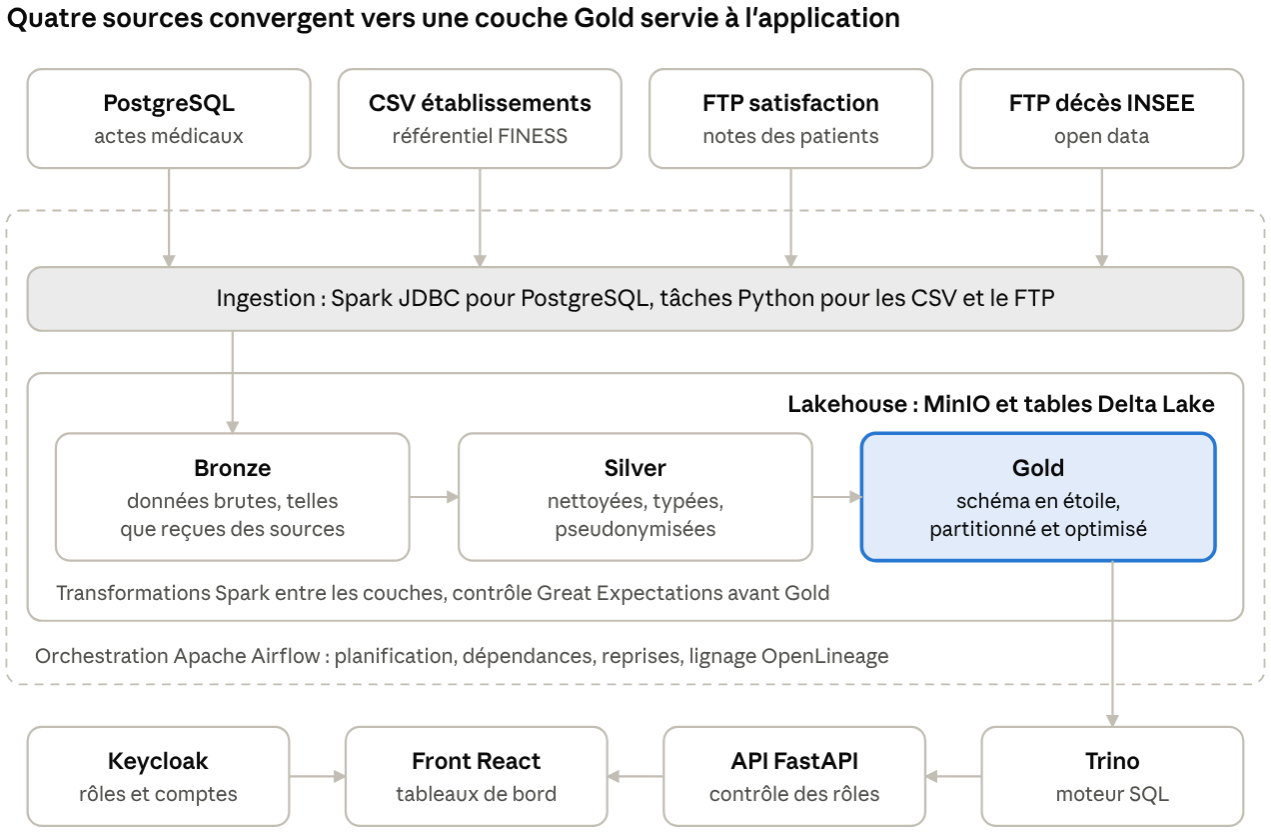
Source	Nature	Contenu	Accès	Usage dans le modèle
S1 Base opérationnelle	PostgreSQL 14, 13 tables	Patient, Consultation, Diagnostic, Professionnel_de_sante, Specialites, Prescription, Medicaments, Laboratoire, Mutuelle, Adher, Salle, date, AAAA	Connexion JDBC	Faits consultation et hospitalisation, dimensions patient, diagnostic, professionnel
S2 Référentiel établissements	Fichiers CSV	Capacités, certifications, localisation par codes FINESS et INSEE	Fichiers locaux	Dimension établissement et hiérarchie géographique

Source	Nature	Contenu	Accès	Usage dans le modèle
S3 Satisfaction	Fichiers plats via FTP	Notes de satisfaction par établissement	FTP	Fait satisfaction
S4 Décès INSEE	Fichiers plats via FTP, open data	Fichier national des personnes décédées	FTP	Fait décès

Pour chaque source, nous produisons un modèle conceptuel, une volumétrie et une description de chaque attribut. Ce travail alimente directement le dictionnaire de données du livrable 1.

4. Architecture cible et fonctionnement de bout en bout

La plateforme suit le paradigme Lakehouse en trois couches : Bronze, Silver et Gold. Chaque couche a un rôle unique, ce qui rend les traitements lisibles, rejouables et auditables.



architecture cible · sources, Lakehouse, restitution

Les données descendent des sources vers la couche Gold. L'application ne lit jamais que la couche Gold, via Trino.

Le parcours d'une donnée

1. Collecte : Airflow déclenche l'extraction. Spark lit les tables PostgreSQL par JDBC ; des tâches Python récupèrent les CSV et les fichiers déposés sur le FTP.
2. Bronze : les données sont écrites telles quelles dans MinIO, au format Delta, avec la date de chargement et la source d'origine. Rien n'est perdu et tout peut être rejoué.
3. Silver : Spark nettoie, type et dédoublonne les données, harmonise les codes FINESS et INSEE, et pseudonymise les patients. Les lignes invalides partent dans une table de rejets.
4. Contrôle : Great Expectations vérifie complétude, unicité et cohérence. Un contrôle bloquant en échec arrête le pipeline avant la couche Gold.
5. Gold : Spark construit les dimensions et les tables de faits du schéma en étoile, partitionnées et optimisées pour les requêtes analytiques.
6. Restitution : l'application interroge la couche Gold via Trino. Keycloak authentifie l'utilisateur et l'API ne renvoie que les vues autorisées pour son rôle.

Rôle de chaque couche

Couche	Contenu	Règle principale	Utilisateurs
Bronze	Copie brute des sources, historisée	Aucune transformation, ajout seulement	Data engineers
Silver	Données propres, typées, pseudonymisées	Une table par entité métier, sans donnée directement identifiante	Data engineers
Gold	Schéma en étoile et agrégats	Optimisée pour la lecture, seule couche exposée	Application, praticiens, direction

5. Stack technique et justification

Nous retenons une stack open source moderne, centrée sur Apache Spark et Delta Lake, et entièrement déployée avec Docker Compose. Elle reproduit en local l'architecture d'une plateforme data d'entreprise et peut être portée sur un cloud certifié HDS sans réécriture.

Brique	Outil retenu	Rôle	Pourquoi ce choix
Conteneurisation	Docker Compose	Déployer toute la plateforme en une commande	Environnement identique sur les trois postes, fin des problèmes d'installation
Stockage objet	MinIO	Stocker les fichiers des couches Bronze, Silver et Gold	Compatible S3 : le même code fonctionne sur AWS S3, Azure ou OVHcloud
Format de table	Delta Lake	Tables transactionnelles au dessus de Parquet	Transactions ACID, historique des versions (time travel), fusion incrémentale (MERGE), Z-ordering
Traitement	Apache Spark 3.5 (PySpark)	Extraction JDBC, transformations, chargement	Traitement distribué, standard du marché, adapté aux dizaines de millions de lignes du fichier des décès
Catalogue	Hive Metastore	Référencer les tables et leurs schémas	Partagé entre Spark et Trino, une seule définition des tables
Moteur de requête	Trino	Répondre aux requêtes analytiques de l'application	Requêtes SQL interactives en quelques secondes, là où Spark est conçu pour les traitements lourds
Orchestration	Apache Airflow	Planifier les pipelines, gérer dépendances, reprises et alertes	Référence open source, pipelines écrits en Python et versionnés
Qualité des données	Great Expectations	Contrôler complétude, unicité, cohérence à chaque chargement	Produit automatiquement le rapport qualité exigé dans le livrable 2
Lignage	OpenLineage et Marquez	Tracer chaque table jusqu'à sa source	Répond à l'exigence de traçabilité des flux du plan de gouvernance

Brique	Outil retenu	Rôle	Pourquoi ce choix
Restitution	FastAPI, React, ECharts, Leaflet	Application décisionnelle développée par l'équipe	Liberté totale sur l'expérience, les rôles et les cartes, sans licence
Authentification	Keycloak	Comptes, rôles praticien et direction	Gestion des accès standard (OpenID Connect), base de la matrice des droits
Versionnement	Git et GitHub	Code, scripts, pipelines, documentation	Travail en parallèle, historique, revue de code

Pourquoi pas la stack Cloudera et Talend du manuel

- Talend Open Studio a été retiré par Qlik en janvier 2024 : il n'est plus maintenu ni distribué gratuitement.
- La machine virtuelle Cloudera QuickStart 5.12 date de 2017 et n'est plus supportée. Elle exige 8 à 16 Go de RAM pour un seul nœud.
- Hive sur cette version ne gère ni les transactions ACID par défaut ni les formats de table modernes.
- Spark et Delta Lake couvrent les mêmes notions du bloc (partitionnement, bucketing, stockage distribué) avec des outils réellement utilisés en entreprise aujourd'hui.

Correspondance avec les outils d'entreprise

Cette architecture suit la même logique que les plateformes industrielles. Spark tient le rôle d'un ETL comme Informatica : chaque job Spark correspond à un mapping. Airflow tient le rôle d'un ordonnanceur comme OpCon : chaque DAG correspond à un enchaînement de jobs avec ses dépendances, sa planification et ses reprises sur erreur.

6. Modèle décisionnel

La couche Gold suit l'approche de Ralph Kimball : quatre tables de faits partagent des dimensions communes, ce qui forme un schéma en constellation. Une même dimension, comme le temps ou l'établissement, garde la même définition pour tous les faits, ce qui permet de croiser les indicateurs.

Tables de faits

Table de faits	Grain (une ligne représente)	Mesures	Indicateurs servis
fait_consultation	Une consultation	Nombre de consultations, âge du patient à la date de l'acte	1, 2, 6
fait_hospitalisation	Un séjour hospitalier	Nombre de séjours, durée de séjour en jours, âge à l'admission	3, 4, 5
fait_deces	Un décès enregistré par l'INSEE	Nombre de décès, âge au décès	7
fait_satisfaction	Un établissement pour une année d'enquête	Score de satisfaction, nombre de répondants	8

Matrice de bus

La matrice indique quelle dimension est reliée à quel fait. C'est le plan directeur du modèle.

Dimension	Consultation	Hospitalisation	Décès	Satisfaction
dim_temps	Oui	Oui	Oui	Oui
dim_etablissement	Oui	Oui	Non	Oui
dim_localisation	Via l'établissement	Via l'établissement	Oui	Via l'établissement
dim_patient	Oui	Oui	Non	Non
dim_diagnostic	Oui	Oui	Non	Non
dim_professionnel	Oui	Non	Non	Non

Dimensions

Dimension	Clé	Attributs principaux	Source
dim_temps	id_date (AAAAMMJJ)	Date, jour, mois, trimestre, année, jour de la semaine	Générée par le pipeline

Dimension	Clé	Attributs principaux	Source
dim_etablissement	id_etablissement	Code FINESS, raison sociale, catégorie, capacité, clé de localisation	S2
dim_localisation	id_commune (code INSEE)	Commune, département, région	S2, référentiel INSEE
dim_patient	id_patient_pseudo	Sexe, année de naissance, clé de localisation	S1, pseudonymisée
dim_diagnostic	id_diagnostic	Code, libellé, catégorie	S1
dim_professionnel	id_professionnel	Spécialité, établissement de rattachement	S1

Toutes les clés des dimensions sont des clés de substitution générées par le pipeline. La clé métier d'origine est conservée en attribut pour la traçabilité. Le dictionnaire de données complet, avec types et cardinalités, sera finalisé après l'étude détaillée des sources.

7. Pipelines ETL et orchestration

Chaque source dispose de son propre pipeline Airflow, et un pipeline final construit le modèle Gold une fois toutes les couches Silver à jour. Tous les jobs sont idempotents : relancer un job produit le même résultat sans créer de doublons.

Les DAG Airflow

DAG	Source	Fréquence	Étapes
dag_postgres	S1 PostgreSQL	Quotidienne	Extraction JDBC par table vers Bronze, nettoyage et pseudonymisation vers Silver
dag_etablissements	S2 CSV	Mensuelle	Lecture des CSV, normalisation FINESS et INSEE, rattachement régional

DAG	Source	Fréquence	Étapes
dag_satisfaction	S3 FTP	À chaque nouveau fichier (capteur FTP)	Téléchargement, détection du format, chargement Bronze puis Silver
dag_deces	S4 FTP INSEE	Mensuelle	Téléchargement du fichier du mois, parsing à largeur fixe, chargement incrémental
dag_gold	Couches Silver	Après les quatre DAG précédents	Construction des dimensions puis des faits, contrôles qualité, mise à jour des statistiques

Règles de transformation

- Typage : conversion explicite des dates, des numériques et des codes, avec schéma imposé à la lecture.
- Normalisation : codes FINESS sur 9 caractères, codes INSEE sur 5 caractères avec zéros initiaux, libellés en majuscules sans accents pour les jointures.
- Valeurs nulles : un attribut obligatoire nul envoie la ligne en rejet ; un attribut descriptif nul prend la valeur « Non renseigné » et un membre dédié existe dans chaque dimension.
- Doublons : détection sur la clé métier, conservation de l'enregistrement le plus récent, comptage des doublons écartés.
- Dérivées : âge à la date de l'acte, tranche d'âge, région déduite du code commune.
- Chargement : MERGE Delta sur la clé métier, ce qui permet les mises à jour sans rechargement complet.

Gestion des erreurs et des rejets

- Les lignes rejetées sont écrites dans une table de rejets par source, avec le motif, la date et le job d'origine.
- Un job en échec est relancé automatiquement deux fois par Airflow, puis signalé par alerte.
- Si un contrôle Great Expectations bloquant échoue, la couche Gold n'est pas mise à jour : l'application continue d'afficher la dernière version valide.
- Chaque exécution journalise le nombre de lignes lues, écrites et rejetées.

8. Optimisation physique et benchmark

Nous mesurons chaque requête métier deux fois : sur une version naïve des tables, puis sur la version optimisée. L'écart obtenu constitue le cœur du livrable 2.

Stratégie d'optimisation

Technique	Application	Effet attendu
Format colonnaire	Parquet compressé au lieu de CSV	Lecture des seules colonnes utiles, volume disque réduit
Partitionnement	Faits partitionnés par année, décès et satisfaction par année et région	Les filtres sur une période ou une région ne lisent que les partitions concernées
Bucketing	Faits répartis en buckets sur la clé patient et la clé diagnostic	Jointures sans redistribution des données entre exécuteurs
Z-ordering Delta	Tri multidimensionnel sur établissement et diagnostic	Saut des fichiers non pertinents grâce aux statistiques min et max
Compactage	Commande OPTIMIZE sur les tables Delta	Moins de petits fichiers, donc moins d'ouvertures de fichiers
Statistiques	ANALYZE TABLE pour l'optimiseur	Meilleurs plans d'exécution dans Spark et Trino
Broadcast	Diffusion des petites dimensions aux exécuteurs	Jointure en mémoire, sans tri ni redistribution

Protocole de benchmark

1. Charger les mêmes données dans deux versions : CSV non partitionné, puis Delta optimisé.
2. Exécuter au moins six requêtes représentatives des indicateurs (un par indicateur obligatoire, plus les décès par région).
3. Lancer chaque requête cinq fois, écarter la première exécution (cache froid) et retenir la médiane.
4. Comparer les temps sur Spark SQL et sur Trino, et relever le volume de données lu dans les plans d'exécution.
5. Restituer les résultats en graphiques avant et après, avec l'interprétation de chaque gain.

Contrôles qualité

Le rapport qualité du livrable 2 est généré par Great Expectations à chaque chargement. Il couvre le taux de complétude par attribut, l'unicité des clés, la cohérence référentielle entre faits et dimensions, la validité des codes FINESS et INSEE, et le nombre de doublons traités.

Delta Lake ne gère pas de clés étrangères contraignantes. L'intégrité référentielle est donc garantie par le pipeline et vérifiée par ces contrôles, ce que nous documentons explicitement.

9. Application décisionnelle développée par l'équipe

Plutôt qu'un outil BI du marché, nous développons notre propre application web. Elle interroge la couche Gold via Trino et adapte son contenu au rôle de l'utilisateur connecté.

Fonctionnement

1. L'utilisateur se connecte via Keycloak, qui lui attribue un rôle : praticien ou direction.
2. Le front React envoie ses demandes à l'API FastAPI avec le jeton d'authentification.
3. L'API vérifie le rôle, construit une requête SQL paramétrée et l'exécute sur Trino.
4. Les résultats fréquemment demandés sont mis en cache pour un affichage immédiat.
5. Le front affiche graphiques (ECharts), cartes régionales (Leaflet) et tableaux de synthèse.

L'API n'exécute que des requêtes paramétrées prédéfinies : aucun texte saisi par l'utilisateur n'est injecté dans le SQL.

Vues par profil

Vue	Profil	Contenu
Activité de consultation	Praticien et direction	Taux de consultation par établissement, diagnostic et professionnel, filtres par période
Hospitalisations	Praticien	Taux par diagnostic, pyramide par sexe et âge, évolution mensuelle
Pilotage des établissements	Direction	Comparaison des établissements, capacités, satisfaction par région
Mortalité	Direction	Carte des décès par région en 2019, comparaison avec l'activité hospitalière

Vue	Profil	Contenu
Performance technique	Équipe projet	Temps de réponse avant et après optimisation, exécution d'une requête en direct

La vue Performance technique sert directement la démonstration live de la soutenance : elle exécute une requête devant le jury et affiche son temps de réponse.

10. Gouvernance des données, RGPD et HDS

Les données de santé sont des données sensibles au sens de l'article 9 du RGPD. La protection est donc intégrée à chaque couche de l'architecture, et non ajoutée à la fin.

Protection des données personnelles

- Pseudonymisation : l'identifiant patient est remplacé dès la couche Silver par un hash SHA-256 avec un sel secret, stocké hors du dépôt de code.
- Minimisation : noms, prénoms, adresses et numéros de téléphone ne dépassent jamais la couche Bronze.
- Généralisation : la date de naissance devient une année puis une tranche d'âge dans la couche Gold.
- Seuil de diffusion : l'application masque tout résultat portant sur moins de 10 patients, pour éviter la réidentification.
- Conservation : la couche Bronze est purgée selon une durée définie, l'historique Delta est limité par la commande VACUUM.

Rôles et droits d'accès

Rôle	Bronze	Silver	Gold	Application
Data engineer	Lecture et écriture	Lecture et écriture	Lecture et écriture	Toutes les vues
Praticien	Aucun accès	Aucun accès	Via l'application	Vues médicales
Direction	Aucun accès	Aucun accès	Via l'application	Vues de pilotage
Administrateur sécurité	Gestion des comptes	Gestion des comptes	Gestion des comptes	Administration Keycloak

Traçabilité

- Lignage automatique de chaque table jusqu'à sa source avec OpenLineage et Marquez.
- Journaux Airflow pour chaque exécution : dates, volumes, rejets, erreurs.
- Journal d'accès de l'API : qui a consulté quelle vue et quand.
- Historique des versions de chaque table grâce au time travel Delta.

Conformité HDS

L'environnement local du projet n'est pas certifié HDS, ce qui est acceptable dans un cadre pédagogique avec des données fournies par l'école. En production, la plateforme serait déployée chez un hébergeur certifié HDS, comme OVHcloud ou Scaleway. Ces offres proposent un stockage compatible S3 et Kubernetes, ce qui permet de reprendre notre architecture sans modification. Le chiffrement au repos et en transit (TLS) serait alors activé sur toutes les briques.